



UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II

Scuola Politecnica e delle Scienze di Base
Corso di Laurea in Ingegneria Informatica

Elaborato finale in **Networks and Cloud Infrastructures**

Monitoraggio del traffico e contrasto al DoS in reti SDN

Membri:

Simone Valerio matr. M63001820

Alessandro Alberino matr. DE9000116

Introduzione

Nel panorama attuale delle infrastrutture di rete, la tecnologia Software-Defined Networking (SDN) rappresenta un modello chiave grazie alla netta separazione tra il piano di controllo (Control Plane) e il piano dei dati (Data Plane). Questa architettura rende la gestione della rete altamente flessibile e programmabile, ma espone contestualmente il controllore centrale a possibili vulnerabilità, in particolare verso attacchi di tipo Denial of Service (DoS). Nello specifico, attacchi volumetrici basati su protocolli di livello 3 e 4 (come ICMP, TCP e UDP flooding) rischiano di saturare la capacità dei collegamenti e di sovraccaricare le risorse computazionali degli switch e del controller attraverso continue richieste di elaborazione. L'obiettivo di questo progetto è la realizzazione e la validazione sperimentale di un meccanismo di difesa automatico e reattivo basato su Ryu Controller e protocollo OpenFlow v1.3. L'applicazione sviluppata, denominata L4AntiDoS, monitora costantemente i flussi di traffico attivi nella rete calcolando soglie di anomalia dinamiche in funzione della larghezza di banda nominale associata a ciascuna interfaccia. Non appena un flusso eccede la soglia stabilita, il sistema identifica tempestivamente l'host malevolo e applica un blocco globale tramite l'installazione di regole OpenFlow di drop, preservando al contempo il corretto instradamento del traffico lecito.

1: Ambiente di emulazione e strumenti di sviluppo

Per la progettazione, la scrittura del codice e la successiva validazione sperimentale è stato configurato un ambiente composto dai seguenti strumenti:

- Oracle VM VirtualBox: Hypervisor per l'esecuzione di una macchina virtuale Linux (Ubuntu Server);
- Mininet: Ambiente CLI eseguito all'interno della VM per creare ed emulare la topologia con switch logici Open vSwitch (OVS), host virtuali e vincoli di banda sui collegamenti;
- Ryu SDN Framework: Controller Python dedicato all'elaborazione degli eventi OpenFlow 1.3, alla raccolta delle statistiche e alla mitigazione attiva;
- Iperf / D-ITG: Strumenti per la generazione controllata sia del traffico regolare a basso throughput, sia dei flussi anomali ad alto volume usati per simulare l'attacco DoS.

2 Topologia e architettura di rete

L'infrastruttura di rete emulata è costituita da un controllore centrale (Ryu Controller) e da un piano dati distribuito formato da 5 switch logici Open vSwitch (S_1, S_2, S_3, S_4, S_5) interconnessi a 4 nodi terminali (host):

- Host 1 (H1): Nodo adibito alla generazione del traffico malevolo e dell'attacco DoS, connesso allo switch edge di ingresso S_1 ;
- Host 2 (H2): Host autorizzato utilizzato per la trasmissione del traffico lecito continuo, collegato allo switch edge S_2 ;
- Host 3 (H3): Server di destinazione (target/vittima), attestato sullo switch edge di uscita S_5 ;
- Host 4 (H4): Nodo ausiliario collegato allo switch S_4 per la verifica della connettività e dell'inoltro multi-nodo.

3 Generalità

Una delle caratteristiche fondamentali del sistema è l'adozione di un approccio dinamico per la determinazione delle soglie di anomalia dei flussi di pacchetti. L'applicazione rileva i vincoli di banda reali di ciascuna porta e vi adatta la tolleranza in modo proporzionale.

Nello script `topology.py`, la classe custom `MyTopo` popola in modo automatico all'atto dell'istanziamento un file di configurazione in formato JSON salvato nel percorso `/tmp/port_bw.json`.

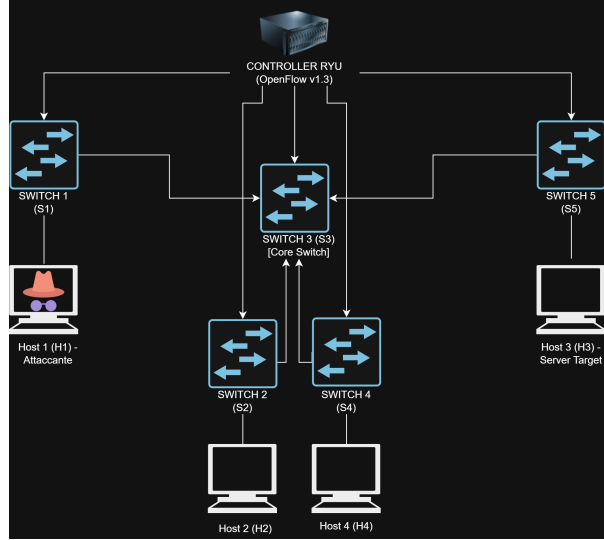


Figure 1: Topologia di rete

Tale file mappa ogni interfaccia della rete associando alla tupla (DPID, Port_No) la rispettiva larghezza di banda nominale espressa in Mbps. All'avvio dell'applicazione, il controller Ryu legge questa mappa per determinare i tassi limite dei singoli collegamenti.

Nel caso in cui il file risulti assente o inaccessibile, il codice intercetta l'eccezione applicando un valore di salvaguardia (*fallback*) pari a 20 Mbps, garantendo l'esecuzione senza arresti anomali del framework. Il calcolo della soglia massima di pacchetti ammissibili per ciascun intervallo di campionamento avviene in questo modo:

- **Conversione della capacità di collegamento:** la larghezza di banda nominale BW_{Mbps} viene convertita nella frequenza elementare in bit al secondo:

$$BW_bps = BW_Mbps \times 1\,000\,000$$

- **Frequenza teorica di pacchetti al secondo:** si stabilisce che una singola tipologia di traffico o flusso malevolo non possa assorbire più di una quota prefissata della banda disponibile. Tale limite è definito dal parametro $Threshold_Ratio = 0.50$ (50% della capacità del canale). Ipotizzando una dimensione media del frame Ethernet a pieno carico pari a $Bits_Per_Pkt = 12000$ bit (1500 Byte di MTU), il tasso teorico si calcola come:

$$Pkts_Per_Sec = \frac{BW_bps \times Threshold_Ratio}{Bits_Per_Pkt}$$

- **Determinazione della soglia discreta per ciclo di polling:** Poiché il thread asincrono del controller monitora i contatori di flusso ogni $Monitor_Interval = 0.5$ secondi, la soglia discreta di scarto ($Threshold$) per l'isolamento dell'attaccante risulta:

$$Threshold = Pkts_Per_Sec \times Monitor_Interval$$

4 Funzionamento

L'esecuzione del sistema richiede l'attivazione preliminare di due terminali indipendenti:

- **Terminale 1 (Piano di Controllo):** Si avvia il controller Ryu con il modulo di rilevamento DoS;
- **Terminale 2 (Piano Dati):** Si avvia Mininet istanziando la topologia custom definita nella classe MyTopo con protocollo OpenFlow 1.3

```

*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4
h2 -> h1 h3 h4
h3 -> h1 h2 h4
h4 -> h1 h2 h3
*** Results: 0% dropped (12/12 received)
mininet>

```

Figure 2: Pingall

Prima di avviare flussi di carico, si testa la raggiungibilità completa tra tutti gli host mediante il comando pingall all'interno della CLI di Mininet:

L'esito privo di perdite (0% dropped, 12/12 pacchetti ricevuti) conferma che i cinque switch Open vSwitch hanno instaurato correttamente la sessione OpenFlow 1.3 con il controller Ryu e che le tabelle di inoltro L2 garantiscono la comunicazione tra tutti gli host passando per lo switch centrale di Core (S_3).

Per verificare il corretto instradamento e accertare che il traffico a basso volume non generi falsi positivi, viene inviata una sequenza controllata di 4 pacchetti ICMP dall'host h2 verso il server h3.

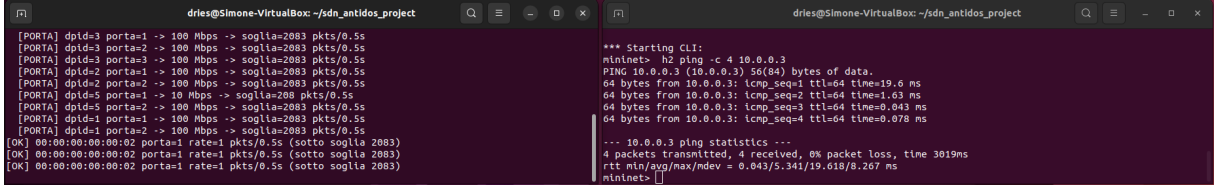


Figure 3: Invio controllato di pacchetti ICMP da H2 verso H3

A destra, il comando `h2 ping -c 4 10.0.0.3` attesta la trasmissione e ricezione corretta dei 4 pacchetti ICMP con perdita nulla (0% packet loss). A sinistra, il monitor di Ryu rileva un tasso pari a un solo pacchetto per intervallo (`rate=1 pkts/0.5s`), valore ampiamente inferiore alla soglia di sicurezza associata alla porta (2083 pkts/0.5s). Di conseguenza, il controller classifica il traffico come lecito e non applica alcuna regola di restrizione.

Per simulare l'attacco DoS mantenendo H_1 come sorgente malevola e H_3 come server bersaglio, viene impiegato iperf con datagrammi compatti da 64 byte a 50 Mbps per 15 secondi.

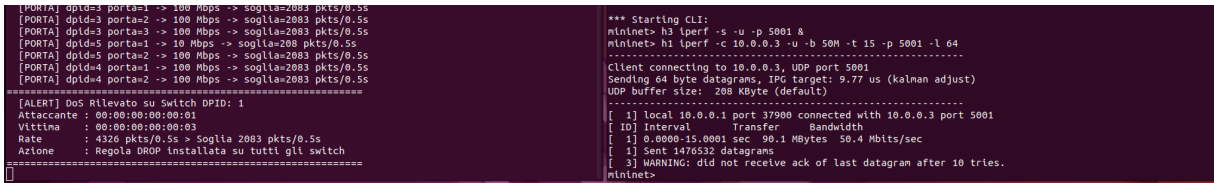


Figure 4: Simulazione dell'attacco DoS con iperf e attivazione del meccanismo di ban sul controller Ryu.

Il monitor di Ryu registra un picco di traffico pari a 4326 pkts/0.5s, superando nettamente la soglia di sicurezza associata alla porta dello switch S_1 (2083 pkts/0.5s). Il controller identifica immediatamente la sorgente dell'attacco (00:00:00:00:00:01) e la vittima (00:00:00:00:00:03), installando proattivamente su tutti i cinque switch della rete le regole OpenFlow di tipo DROP a priorità 100 per neutralizzare il flusso malevolo.

A seguito dell'attivazione del ban, viene eseguito il comando pingall per verificare l'effettivo isolamento dell'attaccante e la persistenza della connettività per gli host legittimi.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> X X X
h2 -> X h3 h4
h3 -> X h2 h4
h4 -> X h2 h3
*** Results: 50% dropped (6/12 received)
mininet>
```

Figure 5: Pingall post-mitigazione.

L'esecuzione del comando `pingall` mostra che l'host attaccante H_1 risulta completamente isolato: ogni tentativo di invio o ricezione da parte sua fallisce (X). Al contempo, gli host leciti (H_2 , H_3 , H_4) mantengono la piena raggiungibilità reciproca. Il tasso di scarto del 50% attesta la selettività della regola DROP, che confina la sorgente malevola senza degradare la connettività del resto della rete.